PROMPT ENGINEERING

WAFT | Crafting Agent Prompts

Overview

System prompts define agent behavior. Well-crafted prompts lead to better Scint scores and more effective evolution.

Prompt Structure

```
1. ROLE      - Who is the agent?
2. CONTEXT   - What situation is it in?
3. TASK      - What should it do?
4. RULES     - What constraints apply?
5. FORMAT    - How should it respond?
```

Example Prompt

```
system_prompt = """
# Role
You are a senior software engineer specializing in Python.

# Context
You work at a company that values clean, tested code.

# Task
Review code for bugs, security issues, and style problems.

# Rules
- Be specific about issues found
- Suggest fixes, don't just criticize
- Prioritize security over style

# Format
Respond in markdown with sections for:
- Summary (1-2 sentences)
- Issues (bulleted list)
- Recommendations
"""
```

Prompt Patterns

The Persona Pattern

Give the agent a specific identity:

```
"You are Dr. Sarah Chen, a quantum physicist who survived
the Quantum Incident. You speak from experience about
the dangers of reality fractures."
```

The Chain-of-Thought Pattern

Encourage step-by-step reasoning:

```
"Before answering, think through the problem step by step.
Show your reasoning process, then provide your conclusion."
```

The Validation Pattern

Build in self-checking:

```
"After generating your response, review it for:
1. Factual accuracy
2. Logical consistency
3. Completeness
If issues are found, revise before submitting."
```

Scint-Aware Prompts

Add explicit Scint avoidance:

```
system_prompt = """
# Scint Prevention Rules
- SYNTAX: Always validate JSON/code before output
- LOGIC: Check for contradictions in your reasoning
- SAFETY: Never generate harmful content
- FACTS: Only state things you're confident about

If uncertain, say "I'm not sure" rather than guess.
"""
```

Evolution-Friendly Prompts

Design prompts that can be mutated:

Good	Bad
Modular sections that can be swapped or modified independently.	Monolithic text that breaks if any part changes.

Testing Prompts

```
# Test prompt against Scint scenarios
waft prompt test --file prompt.txt --scenarios all

# Compare prompts
waft prompt compare prompt_v1.txt prompt_v2.txt

# Analyze prompt
waft prompt analyze --file prompt.txt
```